

Лабораторная работа №3. Редактор геометрических преобразований

В предыдущих работах мы меняли **значения** пикселей (цвет, яркость), но их **координаты** (x, y) оставались на месте. В данной лабораторной работе мы займемся геометрией. Мы будем перемещать пиксели, растягивать изображение, крутить его и искажать.

Математически любое геометрическое преобразование — это функция, которая переводит старые координаты (x, y) в новые (x', y') :

Формула №1. Формула преобразования старых координат в новые.

$$\begin{cases} x' = f_x(x, y) \\ y' = f_y(x, y) \end{cases}$$

где x', y' - значение новой координаты;

$f(x, y)$ - применяемая функция.

Аффинные преобразования

В мире компьютерной графики существует тысячи способов исказить изображение. Но среди них существует стандарт в виде **аффинных преобразований**.

Главное правило аффинности заключается в том что **параллельные линии после преобразования остаются параллельными**. Если вы возьмете решетку (сетку пикселей) и примените к ней аффинное преобразование, ячейки могут растянуться, повернуться или скоситься, превратившись в ромбы или параллелограммы. Но они **никогда** не превратятся в трапеции (как в перспективе) и не изогнутся (как в эффекте "рыбий глаз").

Все эти преобразования описываются **линейными уравнениями**. Это значит что при преобразовании нет никаких квадратов, кубов или корней (в координатах). Только умножение и сложение. Все это выполняется **быстро**.

Прямое преобразование

Суть геометрических преобразований — перенос пикселей с одного места на другое. Самый очевидный способ сделать это — пройти по каждому пикселю **исходного изображения** и вычислить, куда он должен встать на новом изображении. Но как вы узнаете далее самый очевидный способ не всегда является правильным.

Перемещение

Самое простое аффинное преобразование — это операция **Перемещения (Translation)**, которое заключается в сдвиге координатной плоскости на указанное значение.

Формула №2. Формула прямого перемещения координат

$$\begin{cases} x' = x + d_x \\ y' = y + d_y \end{cases}$$

где x', y' - новое значение координат пикселя;

x, y - изначальное значение координат пикселя;

dx, dy - значение на которое перемещается координата.*

Блок кода №1. Функция переноса изображения с прямым преобразованием.

```
// Алгоритм выполняет перемещение изображения, используя прямое преобразование

double translateX = 50; //Фактор переноса по оси X
double translateY = 50; //Фактор переноса по оси Y

//Создает пустое изображение с размерами исходного
Image<Bgr,byte> translatedImage = sourceImage.CopyBlank();

//Проходим по ИСХОДНОМУ изображению
for (int y_in = 0; y_in < sourceImage.Height; y_in++)
{
    for (int x_in = 0; x_in < sourceImage.Width; x_in++)
    {
        //Вычисляем новую позицию для текущего пикселя.
        int x_out = (int)(x_in + translateX);
        int y_out = (int)(y_in + translateY);

        //Проверяем, не вышли ли мы за границы нового изображения
        if (x_out >= 0 && x_out < sourceImage.Width && y_out >= 0 && y_out <
sourceImage.Height)
        {
            //Копируем пиксель из старого изображения в новое
            translatedImage[y_out, x_out] = sourceImage[y_in, x_in];
        }
    }
}

MainImage.Source = ToBitmapSource(translatedImage);
```

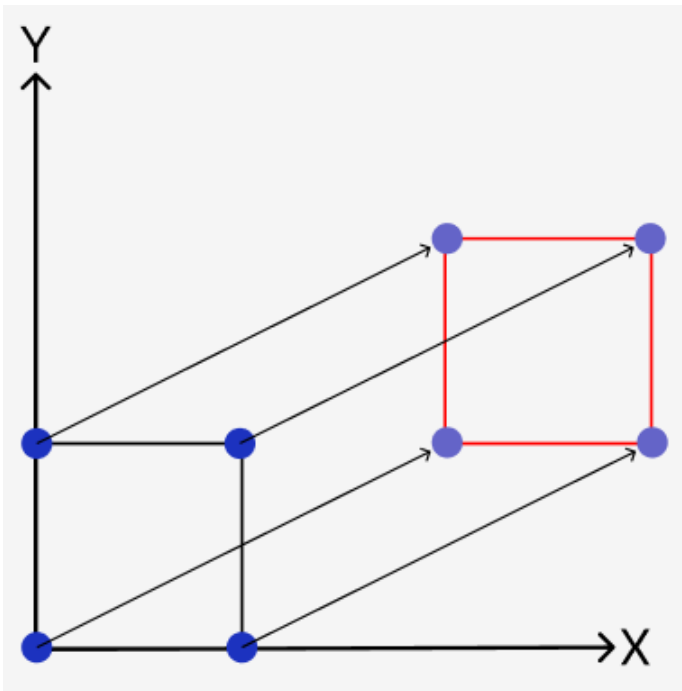


Рисунок №1. Визуализация операции перемещения.

Масштабирование

Другой простой операцией — **Масштабирование (Scaling)**, которое заключается в умножение координаты на коэффициент для получения значения новых координат для пикселя:

Формула №3. Формула прямого масштабирования координат.

$$\begin{cases} x' = x \times S_x \\ y' = y \times S_y \end{cases}$$

где x', y' - значение новой координаты;

x, y - изначальное значение координат;

S_x, S_y - коэффициент масштабирования.

Обратите внимание что прежде чем применять масштабирование требуется так же отмасштабировать изображение в которое будет записываться результат.

Блок кода №2. Масштабирование результирующего изображения `Image<TColor, TDepth>`

```
//Факторы масштабирования
double scaleX = 2;
double scaleY = 2;

//Вычисление нового размера изображения
int newWidth = (int)(sourceImage.Width * scaleX);
int newHeight = (int)(sourceImage.Height * scaleY);

//Создание нового изображения с вычисленными размерами
Image<Bgr, byte> scaledImage = new Image<Bgr, byte>(newWidth, newHeight);
```

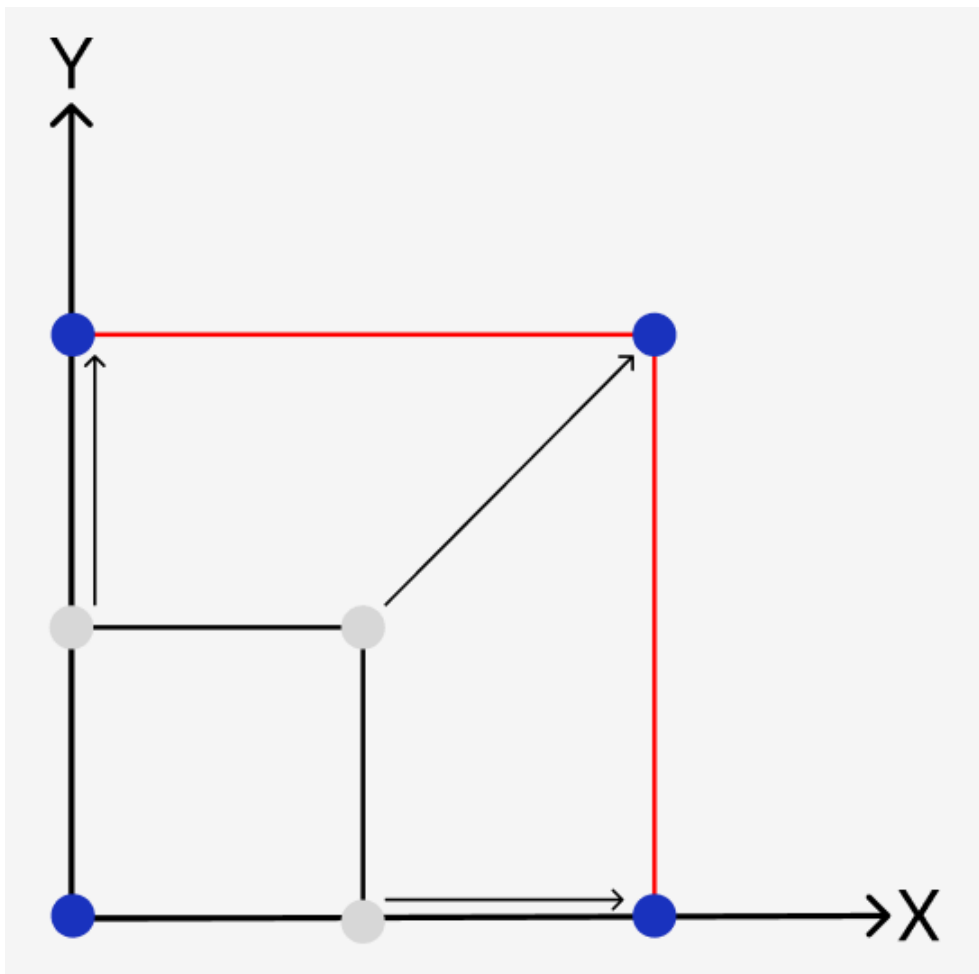


Рисунок №2. Визуализация операции изменения масштаба.

Чем плох данный вариант изменения масштаба? Представьте, что вы увеличиваете картинку в 2 раза ($scaleX$ и $scaleY$ равные двум), с картинкой произойдет следующее:

- Пиксель из позиции $(0, 0)$ останется на том же самом месте;
- Пиксель из позиции $(0, 1)$ встанет на позицию $(0, 2)$.

Вопрос: что будет в позиции $(0, 1)$?

Ответ: ничего. В позиции $(0, 1)$ находится пустота. То как решить эту проблему будет описано далее.

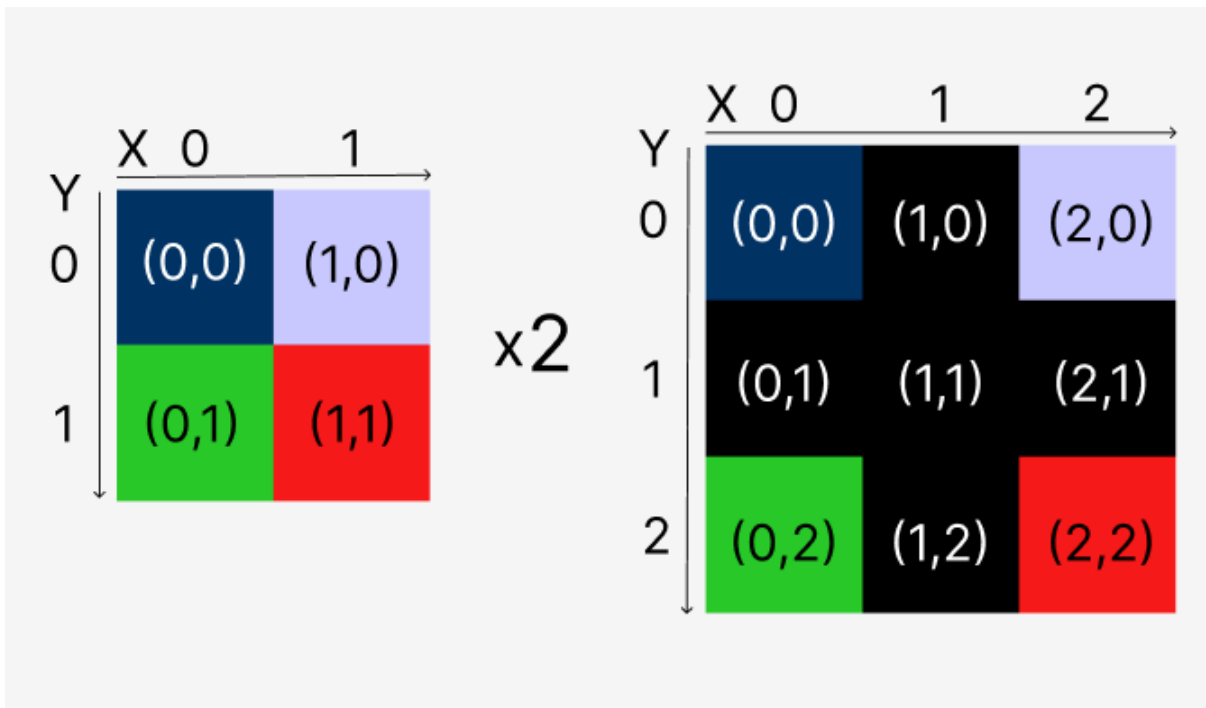


Рисунок №3. Изменение масштаба изображения в два раза с прямым преобразованием.

Скос

Скос (Skew) - это операция по смещению параллельно лежащих линий изображения. В данном случае для каждого из значений координат важна вся координатная плоскость, таким образом что значение координаты x будет зависеть от координаты y и наоборот. Зависимость от перпендикулярной оси координат выражается в том что она определяет на сколько должен сдвинутся каждый новый ряд пикселей.

Формула №4. Формула прямого скоса координат.

$$\begin{cases} x' = x + y \times K_x \\ y' = y + x \times K_y \end{cases}$$

где x', y' - значение новой координаты;

x, y - изначальное значение координат;

K_x, K_y - коэффициент скоса выраженный в радианах.

В результате скоса прямоугольник превращается в **параллелограмм**. Углы перестают быть прямыми (90 градусов), картинка скашивается, но — и это важно для аффинных преобразований — **параллельные линии остаются параллельными**.

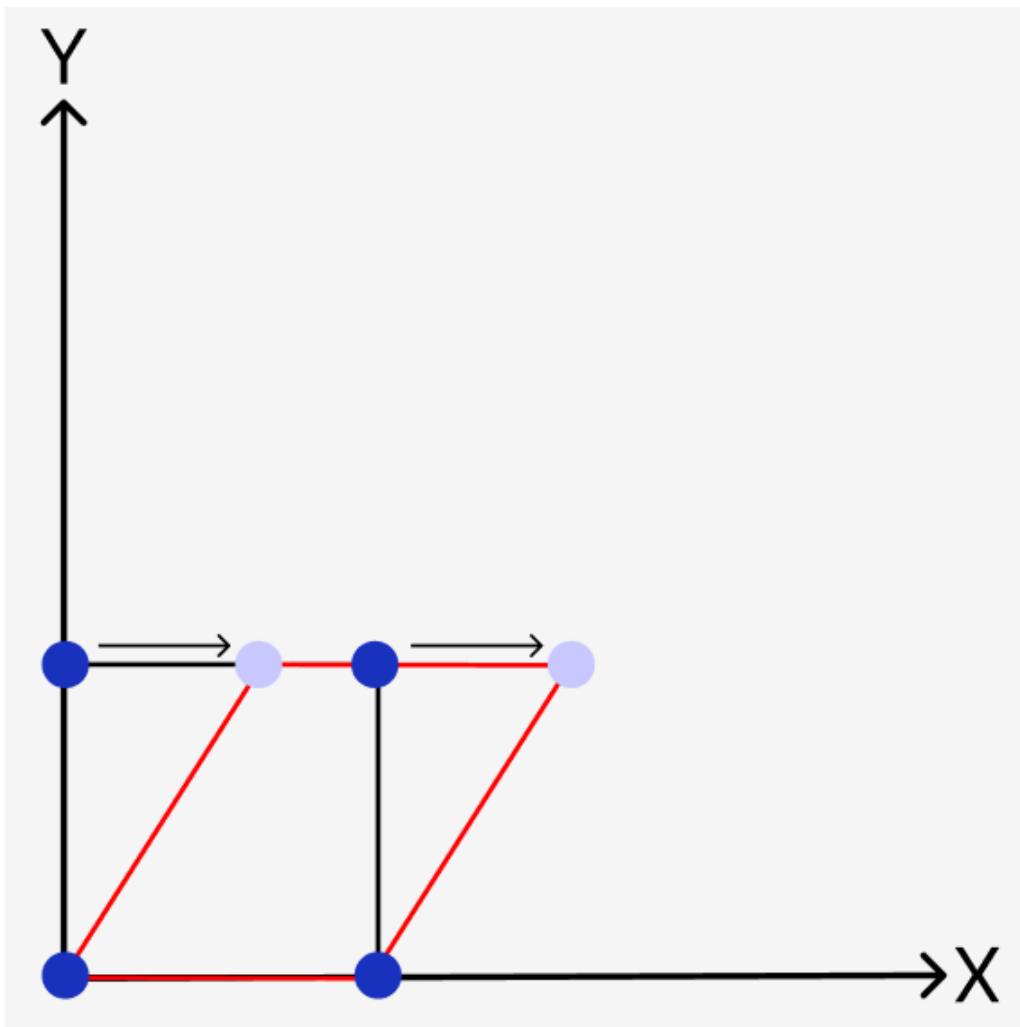


Рисунок №4. Визуализация операции скоса.

Поворот

Поворот (Rotation) — это преобразование, при котором каждая точка изображения перемещается по дуге окружности вокруг фиксированного центра (точки вращения) на заданный угол θ . Как и в случае скоса, новая позиция пикселя зависит от **обеих** исходных координат одновременно. Эта зависимость возникает из-за поворота осей: старая ось X частично проецируется на новую ось Y и наоборот. Математически это выражается через тригонометрические функции синуса и косинуса, полученные из формул суммы углов.

Формула №5. Формула прямого поворота координат.

$$\begin{cases} x' = x \times \cos(\theta) - y \times \sin(\theta) \\ y' = x \times \sin(\theta) + y \times \cos(\theta) \end{cases}$$

где x', y' - значение новой координаты;

x, y - изначальное значение координат;

θ - угол поворота выраженный в радианах.

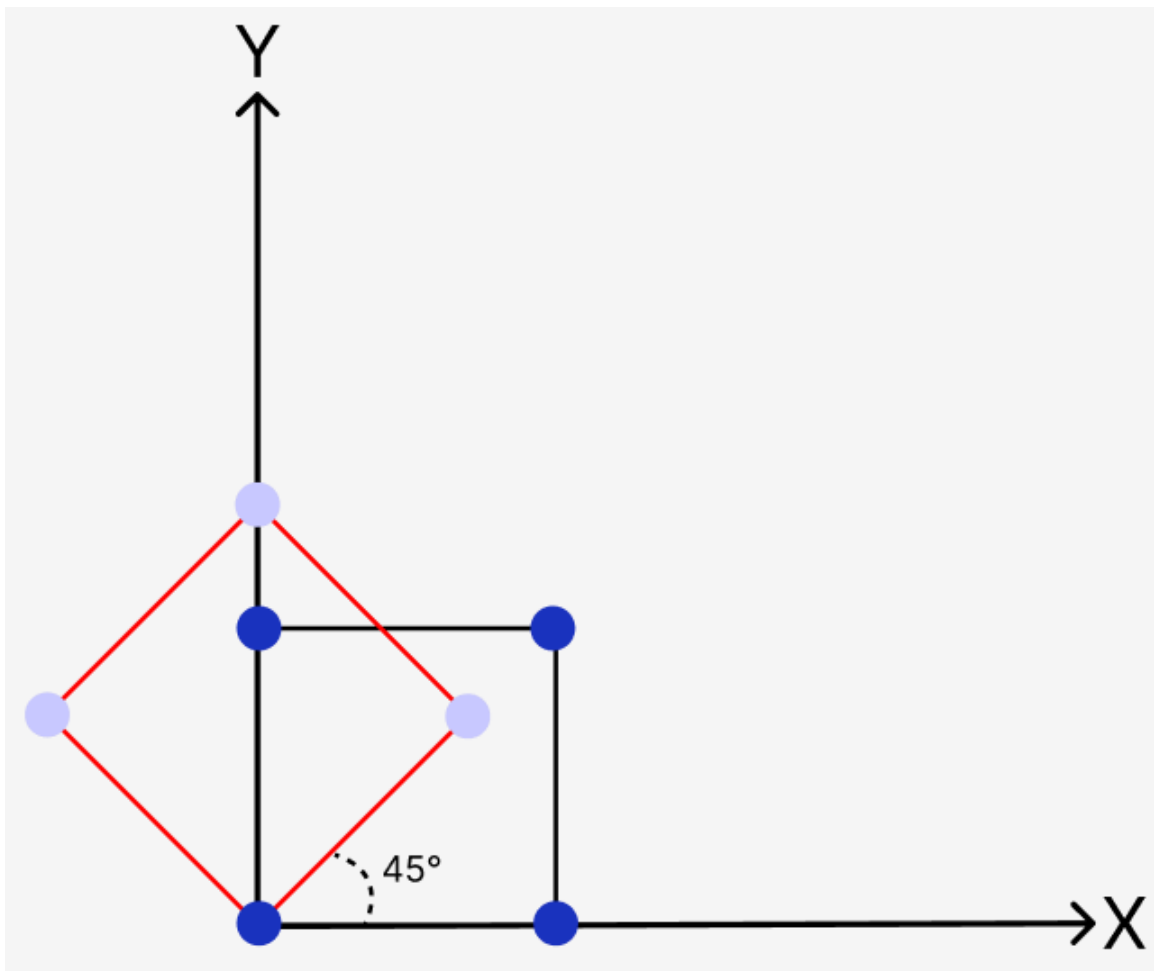


Рисунок №5. Визуализация операции поворота с углом 45 градусов.

В результате поворота изображение изменяет свою ориентацию в пространстве. В отличие от скоса, это **жесткое преобразование**: квадрат остается квадратом, углы между линиями сохраняются (90 градусов), а расстояния между точками не меняются.

Поворот со смещением

Как можно заметить на визуализации и при попытке выполнить эту операцию в программе поворот будет происходить от точки **начала координат**. Это связано с тем что выполняя эту операцию мы по сути поворачиваем всю систему координат в которой находится изображение. Для того чтобы изменить опорную точку поворота можно воспользоваться следующим алгоритмом:

1. Провести операцию перемещения точки которая должна быть опорной в центр системы координат. К примеру если поворот должен происходить от центра изображения, перемещение будет равно половине длины и ширины по соответствующим координатам;
2. Совершаем операцию поворота;
3. Перемещаем изображение обратно с вычисленными ранее коэффициентами смещения.

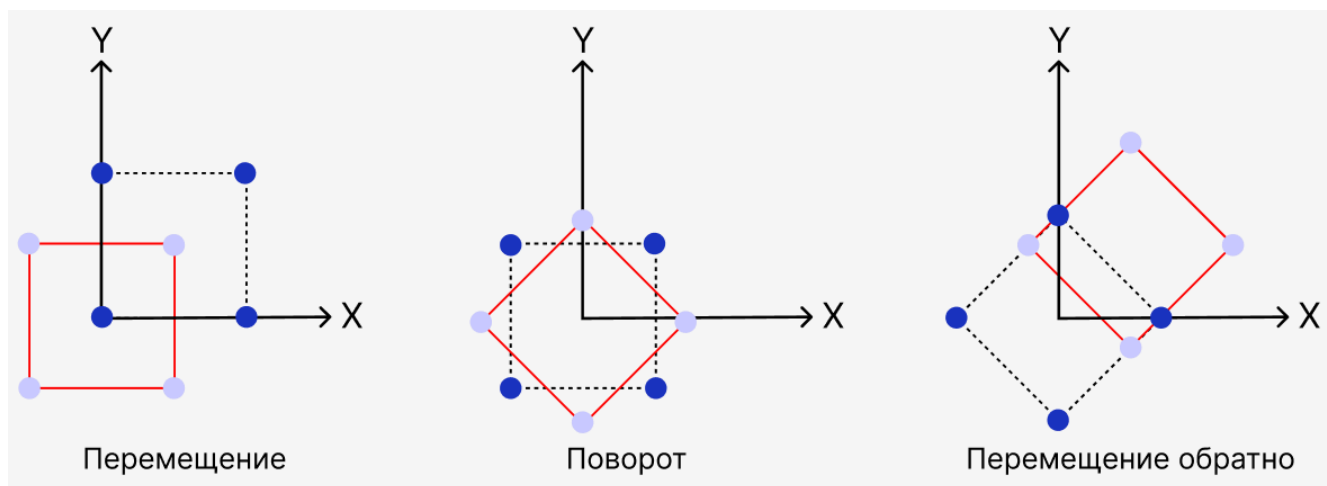


Рисунок №6. Визуализация алгоритма поворота со смещением.

Задача №1

Реализуйте функционал который позволяет выполнить следующие операции над изображением:

- Перемещение изображение;
- Изменение масштаба изображения;
- Скос изображения;
- Поворот изображения вокруг центра изображения;

Важно. Получаемый результат будет зависеть от порядка выполняемых операций (как в примере с поворотом со смещением). Рекомендуется использовать порядок операций в соответствии с тем как они приводятся в методическом пособии:

Перемещение→Масштабирование→Скос→Поворот.

Фильтрация изображения с прямым преобразованием

В данном разделе рассматриваются методы устранения этих артефактов в рамках прямого подхода. Основной задачей этих алгоритмов является заполнение образовавшихся пустот, либо за счет увеличения площади проецируемого пикселя (**Splatting**), либо посредством анализа и интерполяции значений соседних точек на финальном этапе обработки.

Усредняющий фильтр

Самым простым вариантом решения возникшей проблемы будет простой алгоритм который пройдет по всему изображению и закрасит черные пиксели, его можно выразить следующим образом:

1. Находим черный пиксель (значение RGB равно $(0, 0, 0)$);
2. Находим значения ближайших к нему пикселей (сверху, справа, снизу и слева);
3. Для каждого из каналов черного пикселя вычисляем новое значение равное сумме каналов соседних пикселей, разделенное на количество взятых ближайших пикселей;
4. Записываем новый пиксель в изображение.

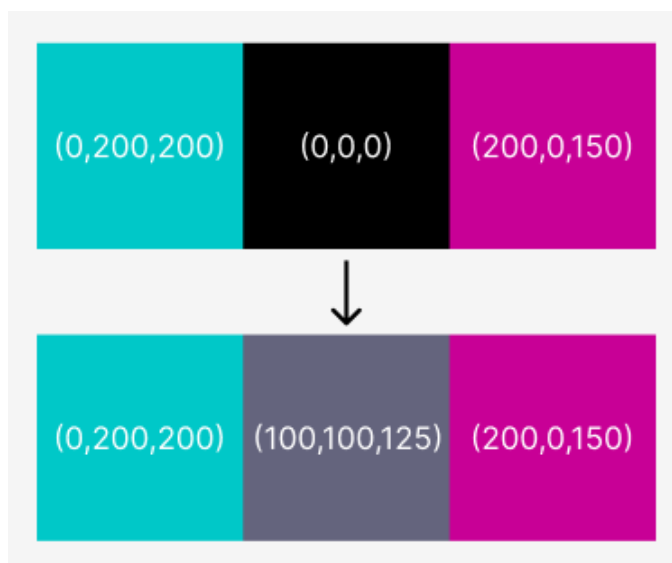


Рисунок №7. Пример определения нового цвета для "пустого" пикселя.

Это бы не был наивный подход если бы в нем не было проблем. Предположим что мы отмасштабировали наше изображение в ширину в пять раз. Теперь у нас есть линия из трех "пустых" пикселей. Теперь у первого найденного пустого пикселя одним из соседей будет другой пустой пиксель, что в свою очередь сделает его гораздо более тусклым чем он должен быть. Та же ситуация будет с последующими пикселями, потому что для них будет соседом не только пустой пиксель, но и закрасенный ранее пиксель отличающийся по насыщенности от основного изображения.

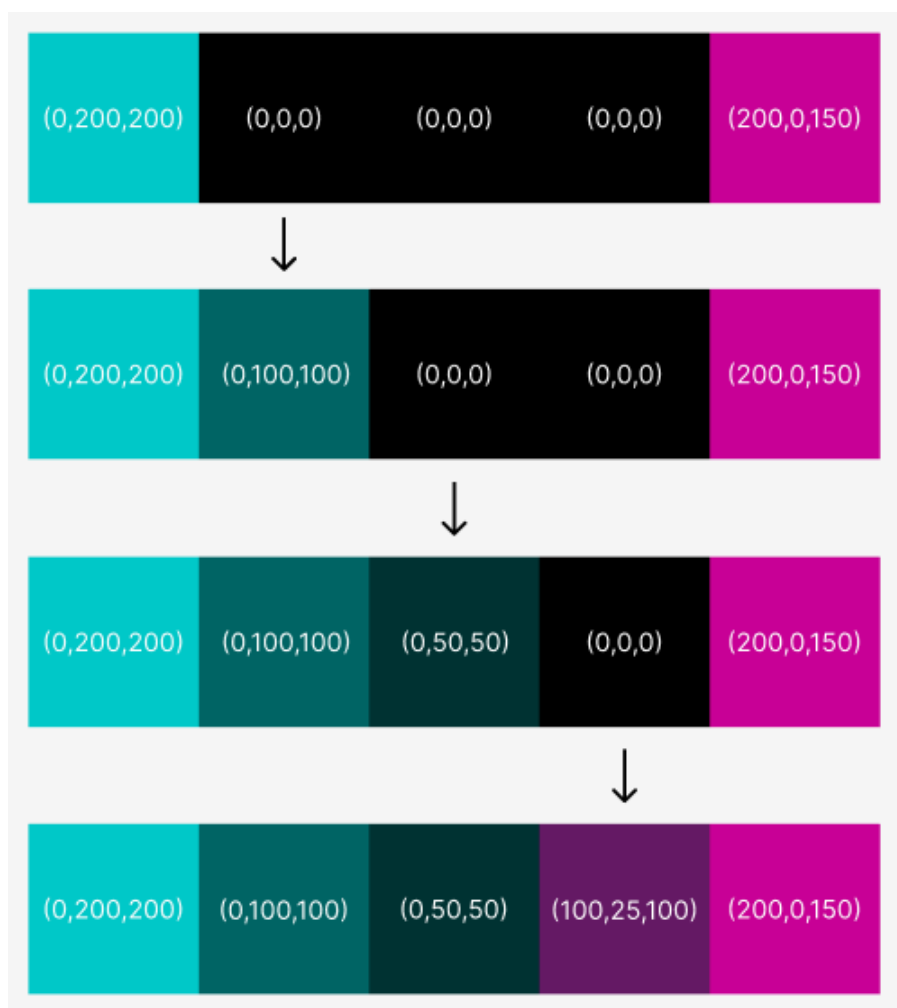


Рисунок №8. Пример проблемы подхода с закрашиванием "пустых" пикселей.

Отмасштабировав изображение и применив к нему фильтр мы все еще будем видеть полосы пикселей, только теперь они будут не черными, а более тусклого цвета чем ближайшие к ним пиксели. С другой стороны данный алгоритм будет корректно обрабатывать для операции поворота, так как в данном случае у нас не добавляется больше пикселей.

Так же важно отметить что данный алгоритм работает через нахождение полностью черных пикселей. Это значит что если изначальное изображение было черным алгоритм будет совершать дополнительные арифметические операции и операции записи что еще сильнее усугубит его производительность.

Расширение пикселя

Вместо того чтобы проходить по выходному изображению и проверять есть ли в нем дырки или нет в данном варианте мы будем заранее определять сколько пикселей будут занимать на выходном изображении пиксели с исходного изображения.

Алгоритм расширения пикселя можно поделить на два этапа. Прежде всего требуется определить сколько будет пикселей на выходной сетке занимать один пиксель исходного изображения.

Блок кода №3. Определение коэффициентов размера пикселя для алгоритма расширения пикселя.

```
//scaleX и scaleY – значения коэффициента масштабирования изображения
//Math.Ceiling используется для округления значения до целого числа
```

```
//В конце рекомендуется добавить единицу для корректной работы с поворотом
int brushWidth = (int)Math.Ceiling(scaleX) + 1;
//Если результат оказался меньше единицы устанавливаем значение кисти на единицу
if (brushWidth < 1) brushWidth = 1;

//Те же операции используем для масштаба по оси Y
int brushHeight = (int)Math.Ceiling(scaleY) + 1;
if (brushHeight < 1) brushHeight = 1;
```

Следующий этап происходит после вычисления нового положения для пикселя. Если значение коэффициента оказалось равным одному это значит что дополнительные пиксели заполнять не требуется. В случае же если коэффициент больше или равен двум мы должны заполнить идущие следом пиксели в соответствии с коэффициентом расширения пикселя.

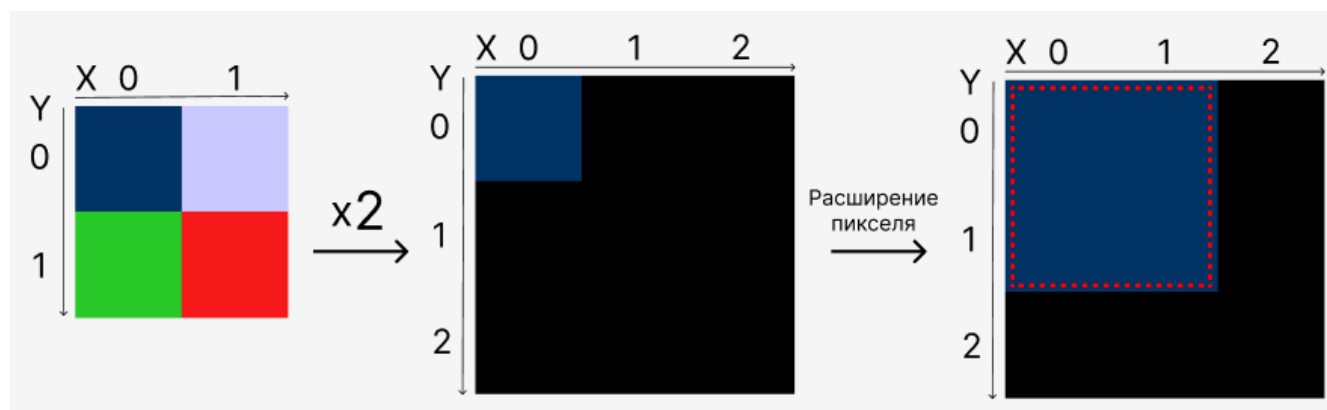


Рисунок №9. Пример работы алгоритма расширения пикселя с масштабированием в два раза.

Задача №2

Реализуйте функционал который дает возможность выбора фильтрации изображения при прямом преобразовании: фильтрация с усредняющим фильтром и фильтрация с расширением пикселя.

Обратное преобразование

В прошлом разделе мы убедились что прямое преобразование это простой путь который имеет множество проблем. Если алгоритм прямого преобразования задает вопрос "куда мне поставить пиксель из старого изображения?", то алгоритм обратного преобразования можно записать как "из какого пикселя взять цвет на оригинале?".

Как можно понять из названия алгоритм **обратного преобразования** работает с измененным изображением производя обратные операции для того чтобы найти на исходном изображении где **примерно** должен был стоять искомый пиксель. Алгоритм можно выразить следующим образом:

1. Создаем пустое итоговое изображение.
2. Мы запускаем цикл **по координатам этого итогового изображения** (x_{out} , y_{out}). Это гарантирует, что мы пройдем по каждому пикселю ровно один раз. Из-за этого на изображении не будет дырок.
3. Для каждого пикселя мы применяем **обратную формулу**:
 - Если изображение должно было быть перемещено ($+dx$, $+dy$), пиксель перемещается в обратную сторону ($-dx$, $-dy$);
 - Если изображение должно было быть увеличено ($*Sx$, $*Sy$), вместо этого координаты делятся на коэффициент масштабирования ($/Sx$, $/Sy$);
 - Если изображение должно было быть повернуто на угол (θ), вместо этого поворот будет производиться в обратную сторону ($-\theta$);
 - Если изображение должно было быть скошено на значение ($+y*Kx$, $+x*Ky$), вместо этого скос происходит в обратную сторону ($-y*Kx$, $-x*Ky$);
4. В результате выполнения обратных операций мы получаем примерные (с дробной частью которую требуется округлить до ближайшего целого) координаты (x_{in} , y_{in}) на оригинальном изображении;
5. После того как был получен оригинальный пиксель (x_{in} , y_{in}) он записывается в пиксель (x_{out} , y_{out}) на итоговом изображении.

Почему обратное преобразование лучше?

Самым очевидным преимуществом обратного подхода по сравнению с прямым является **гарантия полного покрытия**. Это значит что несмотря на количество и масштаб выполняемых операций в изображении никогда не появится дырок. Для того чтобы добиться похожего результата в прямом проходе нам требуются дополнительные функции которые будут заполнять полученные дырки.

Другое важное свойство это современная архитектура графических процессоров. Современная компьютерная графика (видеокарты, шейдеры, OpenGL, DirectX) построена на логике обратного преобразования. Видеокарта запускает **фрагментный шейдер** (о котором мы будем говорить в следующих лабораторных работах) для каждого пикселя экрана используя систему координат UV для поиска цвета на текстуре в этой точке.

Обратное преобразование так же позволяет выполнять операцию **интерполяции**. К примеру в прямом преобразовании мы берем пиксель (10, 10) и помещаем его в координаты (20.5, 20.5). У пикселей не может быть дробной части в координате поэтому мы должны округлить это значение до (21, 21), таким образом мы потеряли точность позиционирования.

В свою очередь в обратном преобразовании исходя из координаты (21, 21) и оказавшись на оригинальном изображении в координате (10.25, 10.25) у нас есть несколько вариантов:

- Мы так же можем округлить это значение и взять ближайший пиксель методом ближайшего соседа;
- Или же мы можем взять значение ближайших соседей и получить новый цвет исходя из пропорций которые мы можем вычислить исходя из дробной части. Тем самым мы реализуем **билинейную фильтрацию**.

Метод ближайшего соседа

Как было описано выше мы получаем **примерные** координаты пикселя с плавающей точкой. Метод ближайшего соседа это самый простой вариант решения проблемы плавающей точки. Для нахождения исходного пикселя требуется округлить значения с плавающей точкой.

Формула №6. Формула получение координаты методом ближайшего соседа.

$$x = \text{Round}(x') \quad y = \text{Round}(y')$$

где x, y - координаты пикселя на исходном изображении

x', y' - координаты пикселя с плавающей точкой

Этот метод работает экстремально быстро, но может давать низкое качество изображения связанное с проблемами **пикселизации** и **алиасинга**.

Билинейная фильтрация

Билинейная интерполяция — это метод вычисления цвета нашей виртуальной точки путем смешивания цветов этих четырех соседей. Вклад каждого соседа в итоговый цвет зависит от расстояния до него: чем ближе точка к соседу, тем весомее его цвет.

Название метода происходит от того, что линейная интерполяция выполняется дважды (bi-linear): сначала вдоль одной оси (например, между парами верхних и нижних пикселей), а затем полученные результаты интерполируются вдоль другой оси.

Алгоритм билинейной фильтрации

Входные данные:

У нас есть дробные координаты искомой точки (x,y) на исходном изображении. В качестве примера возьмем черно-белое изображение и пиксель (5.7, 8.2) .

Шаг 1. Определение соседей (Integer Grid)

Так как пиксели дискретны, точка (5.7, 8.2) лежит внутри квадрата, образованного четырьмя соседними целочисленными координатами. Нам нужно найти их координаты и значения цвета (интенсивности I):

1. Определение верхнего и левого пикселей. Происходит с помощью округления значения вниз:

Формула №7. Формула округления координат для левого-верхнего пикселей.

$$x_1 = \lfloor x \rfloor \quad y_1 = \lfloor y \rfloor$$

где $\lfloor \rfloor$ - операция округления в меньшую сторону

x_1, y_1 - значения для координат левого и верхнего пикселей

Для примера $x_1 = 5, y_1 = 8$

2. Определение нижнего и правого пикселей. Происходит с помощью добавления единицы к координатам рассчитанным в предыдущей формуле:

Формула №8. Формула получения координат нижнего-правого пикселей.

$$x_2 = x_1 + 1 \quad y_2 = y_1 + 1$$

x_2, y_2 - значения для координат нижнего и правого пикселей

*Для примера $x_2 = 6, y_2 = 9$

Цвета в этих точках обозначим как:

- Верхний левый: $Q_{11} = (x_1, y_1)$. Для примера $I = 200$;
- Верхний правый: $Q_{12} = (x_2, y_1)$. Для примера $I = 100$;
- Нижний левый: $Q_{21} = (x_1, y_2)$. Для примера $I = 50$;
- Нижний правый: $Q_{22} = (x_2, y_2)$. Для примера $I = 0$;

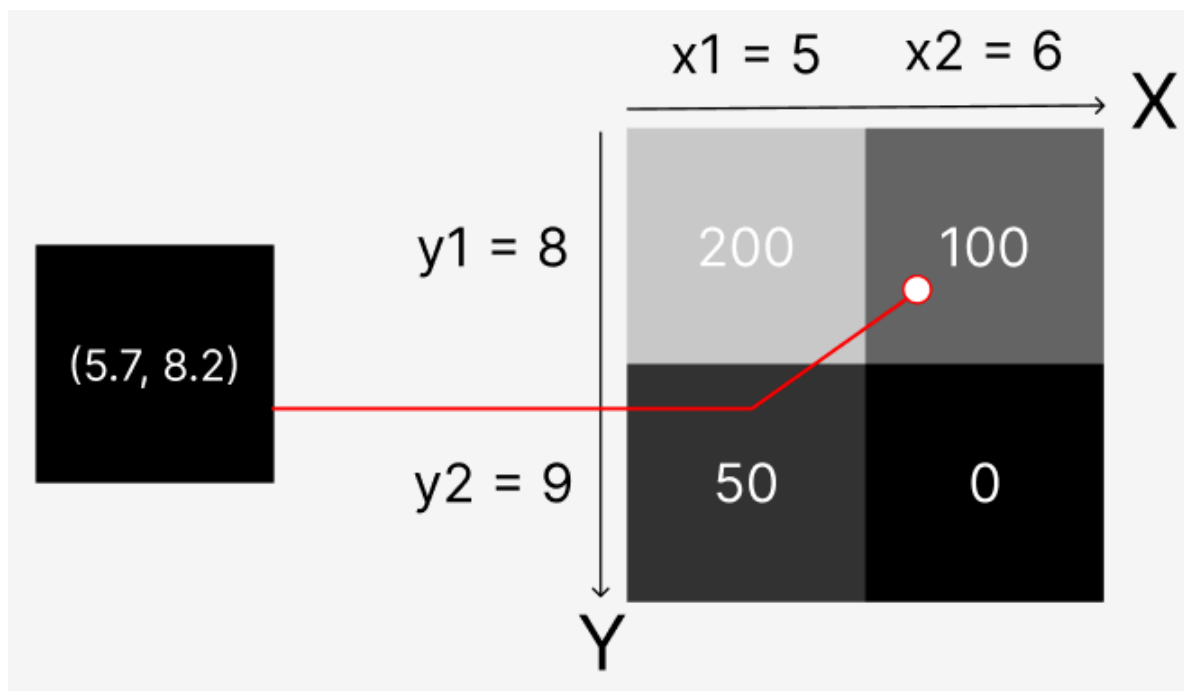


Рисунок №10. Входные данные и полученные координаты пикселей.

Шаг 2. Вычисление весов (Fractional Part)

Нам нужно узнать, насколько близко точка находится к левому и верхнему краю.

Формула №9. Формула расчета весов интерполяции для оси X и Y.

$$dx = x - x_1 \quad dy = y - y_1$$

где dx, dy - веса по соответствующим осям

*Для примера $dx = 0.7$, $dy = 0.2$

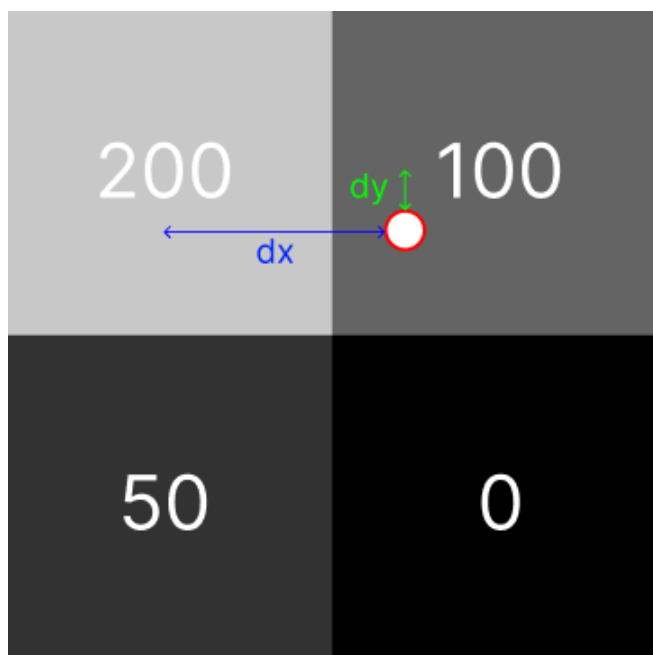


Рисунок №11. Визуализация весов dx и dy .

Веса определяют как пиксели будет влиять на получаемый результат: к примеру если $dx = 0$, значит что искомый пиксель лежит на оси x_1 и значит что пиксели которые лежат на этой оси будут иметь 100% влияние на результирующий цвет. Если же $dx = 0.5$ значит что искомый пиксель лежит между x_1 и x_2 , значит что каждый из пикселей будет иметь одинаковое влияние на результат.

Шаг 3. Линейная интерполяция (Lerp)

Мы сводим задачу к трем простым линейным интерполяциям. Формула линейной интерполяции между значениями R_1 и R_2 :

1. **Интерполируем по горизонтали (верхняя грань).** Находим промежуточное значение R_1 между пикселями Q_{11} и Q_{21} :

Формула №10. Формула расчета значения интерполяции верхней грани пикселей.

$$R_1 = Q_{11} \times (1 - dx) + Q_{21} \times dx$$

Для примера $R_1 = 95$.

2. **Интерполируем по горизонтали (нижняя грань).** Находим промежуточное значение R_2 между пикселями Q_{12} и Q_{22} :

Формула №11. Формула расчета значения интерполяции нижней грани пикселей.

$$R_2 = Q_{12} \times (1 - dx) + Q_{22} \times dx$$

Для примера $R_2 = 30$.

3. **Интерполируем по вертикали (финал).** Находим итоговое значение I между значениями R_1 и R_2 :

Формула №12. Формула расчета итоговой интенсивности.

$$I = R_1 \times (1 - dy) + R_2 \times dy$$

где I - результирующее значение интенсивности.

Для примера интенсивность единственного канала на итоговом изображении будет записано значение 82 .

Важно. Для цветных изображений интерполяция (3 шаг) применяется к каждому из каналов.

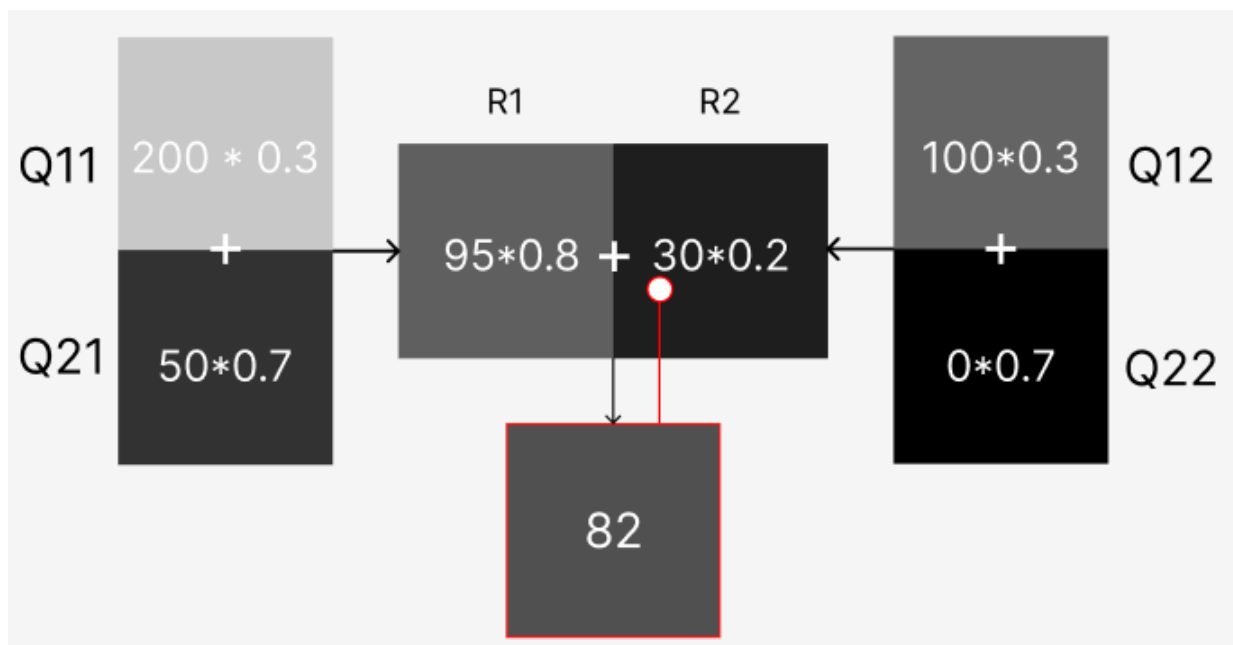


Рисунок №12. Результат интерполяции.

Зачем нужна билинейная фильтрация

Как уже было отмечено ранее, у метода ближайшего соседа существует две проблемы:

Проблема Пикселизации (Blockiness). Если мы просто округлим 10.4 до 10, то все дробные координаты от 9.5 до 10.5 будут ссылаться на один и тот же пиксель. В результате при увеличении изображения мы видим огромные квадратные блоки. Изображение рассыпается на квадраты.

Проблема Алиасинга (Aliasing / "Лесенки"). Если мы поворачиваем изображение, прямые линии становятся наклонными. При использовании округления наклонная линия превращается в "лесенку" из пикселей. Плавные границы становятся рваными и зубчатыми. В динамике (если картинка вращается) это вызывает эффект "дрожания" или "шума" на краях объектов.

Решение этих проблем - билинейная фильтрация. Алгоритм решает эти проблемы, признавая, что точка (10.4, 20.7) находится между четырьмя пикселями. Вместо того чтобы выбирать один пиксель, мы смешиваем цвета всех четырех соседей для получения результата.

Задача №3

Реализуйте функционал который позволяет выполнить операции преобразования изображения используя обратное преобразование. Реализуйте функционал применения билинейной фильтрации к выходному результату

Важно. Так как сами вычисления выполняются в обратном порядке так же должны выполняться и операции. Рекомендуется использовать следующий порядок операций: Обратить поворот→ Обратить скос→ Обратить масштабирование→ Обратить перемещение.

Нелинейные преобразования

Мы научились изменять, масштабировать и крутить наше изображение. Но все эти преобразования были **линейными**, они сохраняют прямые линии прямыми. В этом разделе будут приведены алгоритмы нелинейного изменения геометрии изображения.

Эффект волны

Эффект Волны — это классический пример **нелинейного преобразования**. Мы сдвигаем пиксели не равномерно (как при обычном сдвиге), а по периодическому закону. Чтобы получить волну, идущую, например, вертикально, нам нужно сдвигать каждый пиксель по горизонтали x . Причем величина этого сдвига должна зависеть от вертикальной координаты y .

Формула №13. Формула сдвига оси X с эффектом волны для обратного преобразования.

$$x = x' + A \times \sin(y' \times F)$$

где x - пиксель на основном изображении;

x', y' - пиксели на итоговом изображении;

A - амплитуда волны;

F - частота волн.

Амплитуда отвечает за силу искажения. Это максимальное расстояние (в пикселях), на которое точка сместится влево или вправо. При амплитуде равной нулю ничего не происходит, при амплитуде равной пятидесяти - пиксели будут сдвигаться на пятьдесят пикселей.

Частота отвечает за плотность волн. Это коэффициент, определяющий, как быстро меняется фаза синуса при движении по строкам. Если частота низкая (к примеру 0,05) волна будет длинной и плавной, если высокая (к примеру 0,5) будет частая рябь.

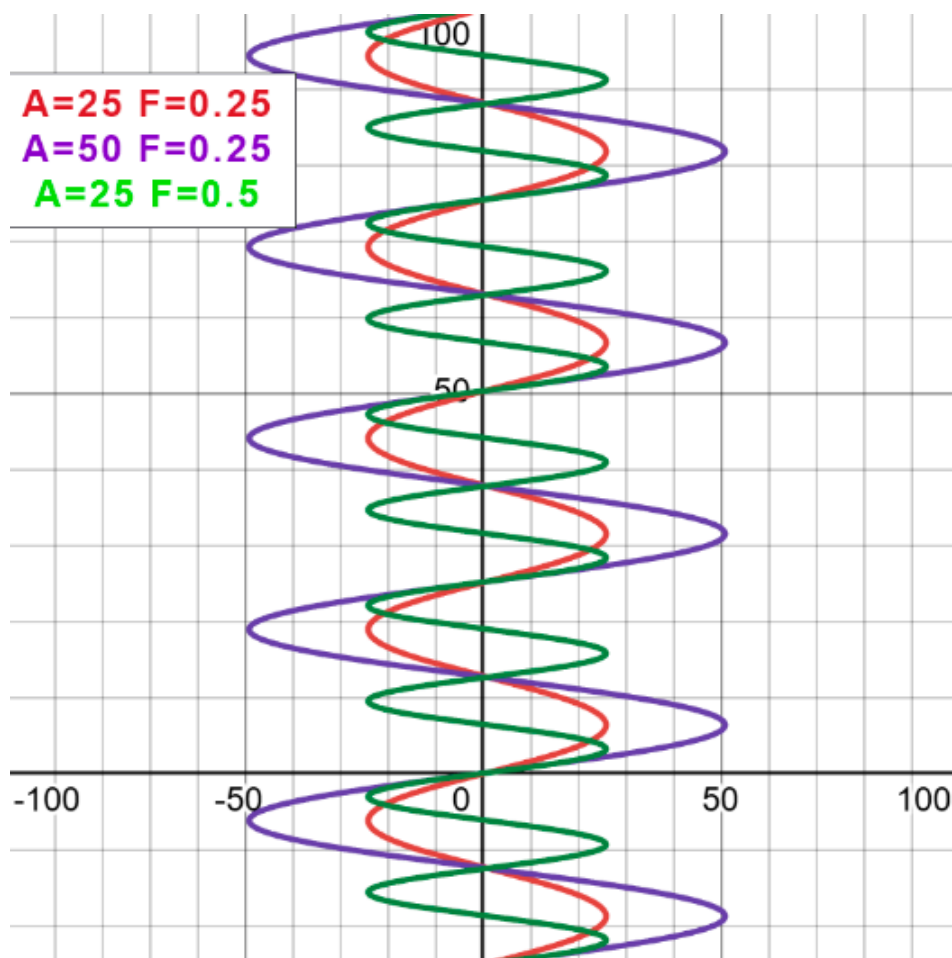


Рисунок №13. График смещения пикселя с координатой $x=1$ в зависимости от различных значений амплитуды и частоты.

Эффект рифленого стекла

Если эффект "Волна" искажает изображение только по одной оси (сдвигает ряды пикселей влево-вправо), то эффект "Стекло" создает хаотичное преломление, искажая координаты и по x , и по y одновременно.

Формула №13. Формула эффекта рифленого стекла для обратного преобразования.

$$x = x' + A \times \sin(y' \times F)$$

$$y = y' + A \times \sin(x' \times F)$$

*где x - пиксель на основном изображении;

x', y' - пиксели на итоговом изображении;

A - амплитуда волны;

F - частота волн.

В результате прямая сетка пикселей превращается в сетку "подушечек" или "пузырей".

Задача №4

Реализуйте функционал который позволяет применить к изображению эффект волны и рифлёного стекла с настраиваемыми параметрами амплитуды и частоты.

Задание на лабораторную работу №3. Редактор геометрических преобразований.

В данной лабораторной работе требуется доработать редактор созданный в предыдущей лабораторной работе добавив в него следующие функции:

1. Функционал изменения изображения (перемещение, масштабирование, скос и поворот) использующий алгоритм прямого преобразования;
2. Функционал фильтрации изображения при его изменении с помощью алгоритмов усредняющего фильтра и расширения пикселя при прямом проходе.
3. 1. Функционал изменения изображения (перемещение, масштабирование, скос и поворот) использующий алгоритм обратного преобразования;
4. Функционал фильтрации изображения при его изменении с помощью алгоритма билинейной фильтрации при обратном преобразовании;
5. Художественные фильтры волны и амплитуды с настраиваемыми параметрами амплитуды и частоты.

Полезные ссылки

1. GitHub репозиторий с учебным проектом: <https://github.com/TheSkyEye1/AOCI-Lab3-Image-Geometry-Editor>
2. GitHub релиз проекта демонстрирующий весь функционал реализуемый в работе: <https://github.com/TheSkyEye1/AOCI-Lab3-Image-Geometry-Editor/releases>
3. Документация Emgu.CV: https://www.emgu.com/wiki/index.php?title=Main_Page
4. Хабр. Аффинные преобразования: <https://habr.com/ru/articles/539420/>
5. CambridgeInColor. Интерполяция в изображениях: <https://www.cambridgeincolour.com/tutorials/image-interpolation.htm>

Дополнительная информация

Бикубическая интерполяция

Мы разобрали Билинейную (по 4 соседям). Но в фотошопе по умолчанию стоит "Бикубическая". Что это такое?

- **Билинейная:** проверяем **4** ближайших пикселя (2×2), строим прямые линии между ними. Результат — гладкий, но немного размытый.
- **Бикубическая:** проверяем **16** ближайших пикселей (4×4), строим сложные кривые (полиномы 3-й степени).

Бикубическая интерполяция дает более четкие границы и меньше размытия, но требует в 4 раза больше вычислений. Это стандарт качества для фотографов, но в играх (в реальном времени) чаще используют билинейную из-за скорости.